



ADVANCED DATABASE SYSTEMS

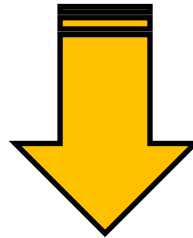
WEEK 9

QUERY OPTIMIZATION PART IV



Query Optimization

For a given SQL Query



Goal:

Build the most “optimal” execution plan for the query



Statistics for Cost Estimation



Cost Estimation

- For each plan considered, must estimate cost:
 - Must **estimate the cost** of each operation in plan tree.
 - ▶ Depends on input cardinalities.
 - Must **estimate the size of result** for each operation in tree!
 - ▶ Use information about the input relations.
 - ▶ For selections and joins, assume independence of predicates.
- We'll discuss the **System R** cost estimation approach.
 - Very inexact, but works ok in practice.
 - More sophisticated techniques known now.



Cost Functions for the JOIN Operation

- Cost functions involve the estimation of file size that results from the JOIN operation
- Join selectivity
 - Ratio of the size of resulting file to size of the CARTESIAN PRODUCT file
 - Simple formula for join selectivity

$$js = |(R \bowtie_c S)| / |(R \times S)| = |(R \bowtie_c S)| / (|R| * |S|)$$

- Join cardinality

$$js = 1 / \max (\text{NDV} (A, R), \text{NDV} (B, S))$$

$$jc = |(R_c S)| = js * |R| * |S|$$



Estimation of the Size of Joins

- If $\text{Schema}(R) \cap \text{Schema}(S) = \{A\}$ is not a key for R or S .
If we assume that every tuple t in R produces tuples in $R \bowtie S$, the number of tuples in $R \bowtie S$ is estimated to be:

$$|R| * \frac{|S|}{NDV(A, S)}$$

If the reverse is true, the estimate obtained will be:

$$|S| * \frac{|R|}{NDV(A, R)}$$

The lower of these two estimates is probably the more accurate one.

$$\frac{|R| * |S|}{\max(NDV(A, R), NDV(A, S))}$$

- Can improve on above if histograms are available
 - Use formula similar to above, for each cell of histograms on the two relations



Estimation of the Size of Joins (Special Cases)

- The Cartesian product $R \times S$ contains $|R| * |S|$ tuples; each tuple occupies $s_r + s_s$ bytes.
- If $\text{Schema}(R) \cap \text{Schema}(S) = \emptyset$, then $R \bowtie S$ is the same as $R \times S$.
- If $\text{Schema}(R) \cap \text{Schema}(S)$ is a key for R , then a tuple of S will join with at most one tuple from R
 - therefore, the number of tuples in $R \bowtie S$ is no greater than the number of tuples in S .
- If $\text{Schema}(R) \cap \text{Schema}(S)$ is a foreign key in S referencing R , then the number of tuples in $r \bowtie s$ is exactly the same as the number of tuples in S .
 - ▶ The case for $\text{Schema}(R) \cap \text{Schema}(S)$ being a foreign key in R referencing S is symmetric.



Estimation of the Size of Joins – General Case

```
SELECT attribute list  
FROM relation list  
WHERE term1 AND ... AND termk
```

- Consider a query block:
- Maximum # tuples in result is the product of the cardinalities of relations in the FROM clause.
- *Selectivity* associated with each *term* reflects the impact of the *term* in reducing result size.
- *Result cardinality* = Max # tuples * product of all selectivities.
 - Implicit *assumption* that *terms* are independent!
 - Term $col=value$ has selectivity $1/NDV(col)$
 - Term $col1=col2$ has selectivity $1/MAX(NDV(col1), NDV(col2))$
 - Term $col>value$ has selectivity $(High(col)-value)/(High(col)-Low(col))$



Cost Functions for the JOIN Operation (cont'd.)

- J1: Page/Block-based Nested-loop join

- For three memory buffer blocks:

$$C_{J1} = b_R + (b_R * b_S) + ((js * |R| * |S|)/bfr_{RS})$$

- For n_B memory buffer blocks:

$$C_{J1} = b_R + (\lceil b_R/(n_B - 2) \rceil * b_S) + ((js * |R| * |S|)/bfr_{RS})$$

- J2: Index-based nested-loop join

- For a secondary index with selection cardinality s_B for join attribute B of S:

$$C_{J2a} = b_R + (|R| * (x_B + 1 + s_B)) + ((js * |R| * |S|)/bfr_{RS})$$



Cost Functions for the JOIN Operation (cont'd.)

- J3: Sort-merge join

- For files already sorted on the join attributes

$$C_{J3a} = b_R + b_S + ((js * |R| * |S|)/bfr_{RS})$$

- Cost of sorting must be added if sorting needed

- J4: Partition-hash join

$$C_{J4} = 3 * (b_R + b_S) + ((js * |R| * |S|)/bfr_{RS})$$



Example: Cost-Based Query Optimization

```
SELECT Pnumber, Dnum, Lname, Address, Bdate
FROM   PROJECT, DEPARTMENT, EMPLOYEE
WHERE  Dnum=Dnumber AND Mgr_ssn=Ssn AND
       Plocation='Stafford';
```

(a)

Table_name	Column_name	Num_distinct	Low_value	High_value
PROJECT	Plocation	200	1	200
PROJECT	Pnumber	2000	1	2000
PROJECT	Dnum	50	1	50
DEPARTMENT	Dnumber	50	1	50
DEPARTMENT	Mgr_ssn	50	1	50
EMPLOYEE	Ssn	10000	1	10000
EMPLOYEE	Dno	50	1	50
EMPLOYEE	Salary	500	1	500

(b)

Table_name	Num_rows	Blocks
PROJECT	2000	100
DEPARTMENT	50	5
EMPLOYEE	10000	2000

(c)

Index_name	Uniqueness	Blevel*	Leaf_blocks	Distinct_keys
PROJ_PLOC	NONUNIQUE	1	4	200
EMP_SSN	UNIQUE	1	50	10000
EMP_SAL	NONUNIQUE	1	50	500

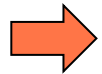
*Blevel is the number of levels without the leaf level.

Sample statistical
information for
relations in the query.
(a) Column statistics
(b) Table statistics
(c) Index statistics



Example: Cost-Based Query Optimization

- Assume optimizer considers only left-deep trees
- Evaluate potential join orders – avoiding cartesian product



- PROJECT ⋈ DEPARTMENT ⋈ EMPLOYEE
- DEPARTMENT ⋈ PROJECT ⋈ EMPLOYEE
- DEPARTMENT ⋈ EMPLOYEE ⋈ PROJECT
- EMPLOYEE ⋈ DEPARTMENT ⋈ PROJECT



Example: Cost-Based Query Optimization

```
SELECT Pnumber, Dnum, Lname, Address, Bdate
FROM PROJECT, DEPARTMENT, EMPLOYEE
WHERE Dnum=Dnumber AND Mgr_ssn=Ssn AND
Plocation='Stafford';
```

Table_name	Column_name	Num_distinct	Low_value	High_value
PROJECT	Plocation	200	1	200
PROJECT	Pnumber	2000	1	2000
PROJECT	Dnum	50	1	50
DEPARTMENT	Dnumber	50	1	50
DEPARTMENT	Mgr_ssn	50	1	50
EMPLOYEE	Ssn	10000	1	10000
EMPLOYEE	Dno	50	1	50
EMPLOYEE	Salary	500	1	500

- Let's compute the cost of the first join order:

PROJECT ⋈ DEPARTMENT ⋈ EMPLOYEE

- Need to determine:

- Access path to each relation
- Join method for each pair

- Dept:

- No index → Full scan

- Project:

- First do the selection on location. Two options:
 - Full scan or use PROJ_PLOC index
 - PROJ_PLOC index: (unclustered, non-unique as Plocation is not a key)
 - Selectivity: $1/NDV(Plocation)$
 - Cardinality after filtering: $1/200 * 2000 = 10$
 - Estimated cost: 12 block accesses (2 for the ix and 10 for data)

Table_name	Num_rows	Blocks
PROJECT	2000	100
DEPARTMENT	50	5
EMPLOYEE	10000	2000

Index_name	Uniqueness	Blevel*	Leaf_blocks	Distinct_keys
PROJ_PLOC	NONUNIQUE	1	4	200
EMP_SSN	UNIQUE	1	50	10000
EMP_SAL	NONUNIQUE	1	50	500

Full Scan Cost: 100 blocks (Index access is cheaper!)



Example: Cost-Based Query Optimization

- Result will be stored in a TEMP1 table with 1 block.
 - blocking factor $2000/100 = 20$ tuples per block $\rightarrow$ 10 rows fit into a single block
- Nested Loops
 - TEMP1 as outer: $1 + 1 * 5$ (Dept table blocks)
 - Writing the result of the join
 - ▶ Estimating the result of the join
 - Dnumber is the key for dept.
 - Any row of Temp1 will match at most 1 tuple from Dept.
 - Therefore join size is at most the size of Temp1.
 - Assume blocking factor of 5
 - 2 blocks will be written in the result.

```
SELECT Pnumber, Dnum, Lname, Address, Bdate
FROM   PROJECT, DEPARTMENT, EMPLOYEE
WHERE  Dnum=Dnumber AND Mgr_ssn=Ssn AND
       Plocation='Stafford';
```

Table_name	Column_name	Num_distinct	Low_value	High_value
PROJECT	Plocation	200	1	200
PROJECT	Pnumber	2000	1	2000
PROJECT	Dnum	50	1	50
DEPARTMENT	Dnumber	50	1	50
DEPARTMENT	Mgr_ssn	50	1	50
EMPLOYEE	Ssn	10000	1	10000
EMPLOYEE	Dno	50	1	50
EMPLOYEE	Salary	500	1	500

Table_name	Num_rows	Blocks
PROJECT	2000	100
DEPARTMENT	50	5
EMPLOYEE	10000	2000



Join Operation: Running Example

Running example:

$student \bowtie takes$

Catalog information:

- $r_{student} = 5,000$.
- $bfr_{student} = 50$, which implies that
 $b_{student} = 5000/50 = 100$.
- $r_{takes} = 10000$.
- $bfr_{takes} = 25$, which implies that
 $b_{takes} = 10000/25 = 400$.
- $NDV(ID, takes) = 2500$, which implies that on average, each student has taken 4 courses.
 - Attribute ID in $takes$ is a foreign key referencing $student$.
 - $NDV(ID, student) = 5000$ (primary key!)

Estimation of the Size of Joins

□ Reminder:

- If $\text{Schema}(R) \cap \text{Schema}(S)$ in S is a foreign key in S referencing R , then the number of tuples in $R \bowtie S$ is exactly the same as the number of tuples in S .
 - ▶ The case for $\text{Schema}(R) \cap \text{Schema}(S)$ being a foreign key referencing S is symmetric.

- In the example query $student \bowtie takes$, ID in $takes$ is a foreign key referencing $student$
 - hence, the result has exactly r_{takes} tuples, which is 10000



Estimation of the Size of Joins

- Compute the size estimates for *student* ⋈ *takes* without using information about foreign keys:
 - $NDV(ID, takes) = 2500$, and $NDV(ID, student) = 5000$
 - The two estimates are $5000 * 10000/2500 = 20,000$ and $5000 * 10000/5000 = 10000$
 - We choose the lower estimate, which in this case, is the same as our earlier computation using foreign keys.



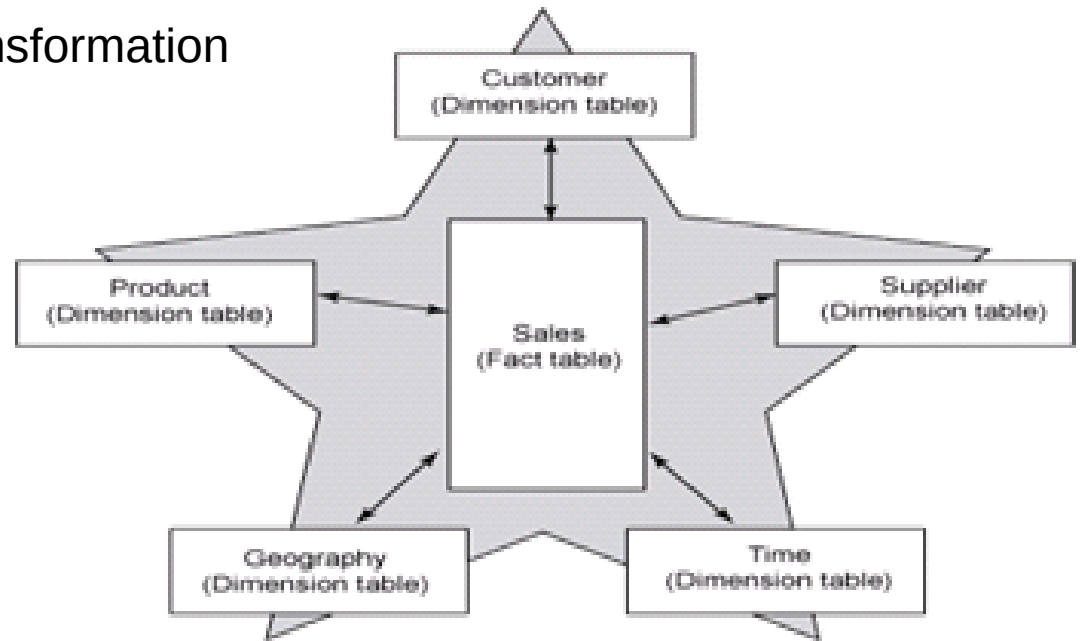
Additional Issues Related to Query Optimization (cont'd.)

- Size estimation of other operations
 - Projection: $|R|$ or $NDV(R)$
 - Set operations:
 - ▶ Union: $|R| + |S|$
 - ▶ Intersection: $\min(|R|, |S|)$
 - Aggregation with Group By
 - ▶ $NDV(C, R)$



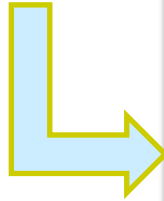
An Example of Query Optimization in Data Warehouses

- ❑ Star transformation optimization
 - ❑ Goal: access a reduced set of data from the fact table and avoid using a full table scan on it
 - ❑ Classic star transformation
 - ❑ Bitmap index star transformation
- ❑ Joining back



Star Transformation

```
SELECT c.cust_city,  
       t.calendar_quarter_desc,  
       SUM(s.amount_sold) sales_amount  
FROM sales s,  
       times t,  
       customers c,  
       channels ch  
WHERE s.time_id = t.time_id  
AND s.cust_id = c.cust_id  
AND s.channel_id = ch.channel_id  
AND c.cust_state_province = 'CA'  
AND ch.channel_desc = 'Internet'  
AND t.calendar_quarter_desc IN ('1999-01','1999-02')  
GROUP BY c.cust_city, t.calendar_quarter_desc;
```



```
SELECT c.cust_city,  
       t.calendar_quarter_desc,  
       SUM(s.amount_sold) sales_amount  
FROM sales s,  
       times t,  
       customers c  
WHERE s.time_id = t.time_id  
AND s.cust_id = c.cust_id  
AND c.cust_state_province = 'CA'  
AND t.calendar_quarter_desc IN ('1999-01','1999-02')  
AND s.time_id IN (SELECT time_id  
                  FROM times  
                  WHERE calendar_quarter_desc IN('1999-01','1999-02'))  
AND s.cust_id IN (SELECT cust_id  
                  FROM customers  
                  WHERE cust_state_province='CA')  
AND s.channel_id IN (SELECT channel_id  
                     FROM channels  
                     WHERE channel_desc = 'Internet')  
GROUP BY c.cust_city, t.calendar_quarter_desc;
```



Additional Optimization techniques



Optimizing Nested Subqueries**

- Nested query example:

```
select name
from instructor
where exists (select *
               from teaches
               where instructor.ID = teaches.ID and teaches.year = 2019)
```

- SQL conceptually treats nested subqueries in the where clause as functions that take parameters and return a single value or set of values
 - Parameters are variables from outer level query that are used in the nested subquery; such variables are called **correlation variables**
- Conceptually, nested subquery is executed once for each tuple in the cross-product generated by the outer level **from** clause
 - Such evaluation is called **correlated evaluation**
 - Note: other conditions in where clause may be used to compute a join (instead of a cross-product) before executing the nested subquery



Optimizing Nested Subqueries

- ❑ Correlated evaluation may be quite inefficient since
 - ❑ a large number of calls may be made to the nested query
 - ❑ there may be unnecessary random I/O as a result
- ❑ SQL optimizers attempt to transform nested subqueries to joins where possible, enabling use of efficient join techniques

```
select name
from instructor
where exists (select *
               from teaches
               where instructor.ID = teaches.ID
               and teaches.year = 2019)
```



```
select name
from teaches, instructor
where instructor.ID = teaches.ID
       and teaches.year = 2019
```

- ❑ Note: the two queries generate different numbers of duplicates (why?)
 - ❑ Can be modified to handle duplicates correctly using semijoins



Optimizing Nested Subqueries

- The **semijoin** operator $\bowtie$ is defined as follows
 - A tuple r_i appears n times in $r \bowtie_{\theta} s$ if it appears n times in r , and there is at least one matching tuple s_i in s

```
select name
from instructor
where ID in (select teaches.ID
                  from teaches
                  where teaches.year = 2019)
```

- The above SQL query can be written in relational algebra as:

$$\Pi_{name}(instructor \bowtie_{instructor.ID=teaches.ID \wedge teaches.year=2019} teaches)$$

- Or even as:

$$\Pi_{name}(instructor \bowtie_{instructor.ID=teaches.ID} (\sigma_{teaches.year=2019} teaches))$$

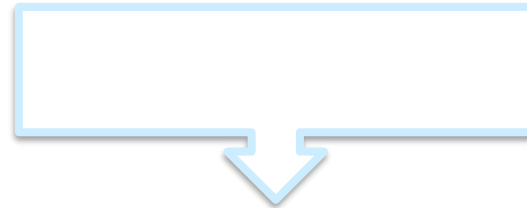
- Now the duplicate count is correct!



Query Rewrite: Unnesting Queries

```
SELECT S.sname  
FROM Sailors S  
WHERE EXISTS  
  (SELECT *  
   FROM Reserves R  
   WHERE R.bid=103  
    AND R.sid=S.sid)
```

Nested block to optimize:
SELECT *
FROM Reserves R
WHERE R.bid=103
AND S.sid= *outer value*



Equivalent non-nested query:
SELECT S.sname
FROM Sailors S, Reserves R
WHERE S.sid=R.sid
AND R.bid=103

□ *The non-nested version of the query is typically optimized better.*



Optimizing Nested Subqueries

- This could also be written using only joins (in SQL) as

```
with  $t_1$  as  
  (select distinct ID  
   from teaches  
   where year = 2019)  
select name  
from instructor,  $t_1$   
where  $t_1.ID$  = instructor.ID
```



Optimizing Nested Subqueries

- The query

```
select name
from instructor
where not exists (select *
                  from teaches
                  where instructor.ID = teaches.ID and teaches.year = 2019)
```

can be rewritten using the **anti-join** operation as $\bar{\bowtie}$

$$\Pi_{name}(instructor \bar{\bowtie}_{instructor.ID=teaches.ID \wedge teaches.year=2019} teaches)$$

The **anti-join** operator $\bar{\bowtie}$:

A tuple r_i appears n times in $r \bar{\bowtie}_{\theta} s$ if it appears n times in r , and there is **no** matching tuple s_j in s



Optimizing Nested Subqueries (Cont.)

In general, SQL queries of the form below can be rewritten as shown

- Rewrite: **select** A
from $r_1, r_2, \dots, r_n$
where P_1 **and exists** (**select** $*$
from $s_1, s_2, \dots, s_m$
where P_2^1 **and** P_2^2)
- To: $\Pi_A(\sigma_{P_1}(r_1 \times r_2 \times \dots \times r_n) \bowtie_{P_2^2} \sigma_{P_2^1}(s_1 \times s_2 \times \dots \times s_m))$
 - P_2^1 contains predicates that do not involve any correlation variables
 - P_2^2 contains predicates involving correlation variables
- The process of replacing a nested query by a query with a join/semijoin (possibly with a temporary relation) is called **decorrelation**.
- Decorrelation is more complicated in several cases, e.g.
 - ▶ The nested subquery uses aggregation, or
 - ▶ The nested subquery is a scalar subquery
- Correlated evaluation used in these cases

Outer Join Size Estimation

□ R FULL OUTER JOIN S:

▶ $|R \text{ join } S| + |R \text{ anti-join } S| + |S \text{ anti-join } R|$



Cost Functions for the JOIN Operation (cont'd.)

- Join selectivity and cardinality for semi-join

```
SELECT COUNT(*)  
FROM T1  
WHERE T1.X IN (SELECT T2.Y  
               FROM T2);
```

- Unnesting query above leads to semi-join

```
SELECT COUNT(*)  
FROM T1, T2  
WHERE T1.X S= T2.Y;
```

- Join selectivity

$$js = \text{MIN}(1, \text{NDV}(Y, T2) / \text{NDV}(X, T1))$$

- Join cardinality

$$jc = |T1|^* js$$



Cost Functions for the JOIN Operation (cont'd.)

- Join selectivity and cardinality for anti-join

```
SELECT COUNT (*)  
FROM T1  
WHERE T1.X NOT IN (SELECT T2.Y  
FROM T2);
```

- Unnesting query above leads to anti-join

```
SELECT COUNT(*)  
FROM T1, T2  
WHERE T1.X A= T2.Y;
```

- Join selectivity

$$js = 1 - \text{MIN}(1, \text{NDV}(T2.y) / \text{NDV}(T1.x))$$

- Join cardinality

$$jc = |T1| * js$$



Materialized Views

- A **materialized view** is a view whose contents are computed and stored.
- Consider the view

```
create view department_total_salary(dept_name, total_salary) as  
select dept_name, sum(salary)  
from instructor  
group by dept_name
```

- Materializing the above view would be very useful if the total salary by department is required frequently
 - Saves the effort of finding multiple tuples and adding up their amounts



Materialized View Maintenance

- The task of keeping a materialized view up-to-date with the underlying data is known as **materialized view maintenance**
- Materialized views can be maintained by recomputation on every update
- A better option is to use **incremental view maintenance**
 - **Changes to database relations are used to compute changes to the materialized view, which is then updated**
- View maintenance can be done by
 - Manually defining triggers on insert, delete, and update of each relation in the view definition
 - Manually written code to update the view whenever database relations are updated
 - Periodic recomputation (e.g. nightly)
 - Incremental maintenance supported by many database systems
 - ▶ Avoids manual effort/correctness issues



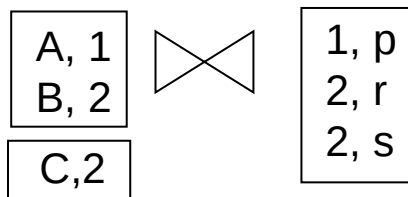
Incremental View Maintenance

- The changes (inserts and deletes) to a relation or expressions are referred to as its **differential**
 - Set of tuples inserted to and deleted from r are denoted i_r and d_r
- To simplify our description, we only consider inserts and deletes
 - We replace updates to a tuple by deletion of the tuple followed by insertion of the update tuple
- We describe how to compute the change to the result of each relational operation, given changes to its inputs
- We then outline how to handle relational algebra expressions



Join Operation

- Consider the materialized view $v = r \bowtie s$ and an update to r
- Let r^{old} and r^{new} denote the old and new states of relation r
- Consider the case of an insert to r :
 - We can write $r^{new} \bowtie s$ as $(r^{old} \cup i_r) \bowtie s$
 - And rewrite the above to $(r^{old} \bowtie s) \cup (i_r \bowtie s)$
 - But $(r^{old} \bowtie s)$ is simply the old value of the materialized view, so the incremental change to the view is just $i_r \bowtie s$
- Thus, for inserts $v^{new} = v^{old} \cup (i_r \bowtie s)$
- Similarly for deletes $v^{new} = v^{old} - (d_r \bowtie s)$



A, 1, p
B, 2, r
B, 2, s
C, 2, r
C, 2, s



Selection and Projection Operations

- Selection: Consider a view $v = \sigma_{\theta}(r)$.
 - $v^{new} = v^{old} \cup \sigma_{\theta}(i_r)$
 - $v^{new} = v^{old} - \sigma_{\theta}(d_r)$
- Projection is a more difficult operation
 - $R = (A,B)$, and $r = \{ (a,2), (a,3) \}$
 - $\Pi_A(r)$ has a single tuple (a) .
 - If we delete the tuple $(a,2)$ from r , we should not delete the tuple (a) from $\Pi_A(r)$, but if we then delete $(a,3)$ as well, we should delete the tuple
- For each tuple in a projection $\Pi_A(r)$, we will keep a count of how many times it was derived
 - On insert of a tuple to r , if the resultant tuple is already in $\Pi_A(r)$ we increment its count, else we add a new tuple with count = 1
 - On delete of a tuple from r , we decrement the count of the corresponding tuple in $\Pi_A(r)$
 - ▶ if the count becomes 0, we delete the tuple from $\Pi_A(r)$



Aggregation Operations

□ **Count** : $v = \gamma_{count(B)}^{(r)}$.

□ When a set of tuples i_r is inserted

- ▶ For each tuple r in i_r , if the corresponding group, $r.A$, is already present in v , we increment its count, else we add a new tuple with count = 1

□ When a set of tuples d_r is deleted

- ▶ for each tuple t in d_r , we look for the group $t.A$ in v , and subtract 1 from the count for the group.
 - If the count becomes 0, we delete from v the tuple for the group $t.A$

□ **Sum**: $v = \gamma_{sum(B)}^{(r)}$

- We maintain the sum in a manner similar to count, except we add/subtract the B value instead of adding/subtracting 1 for the count
- Additionally we maintain the count in order to detect groups with no tuples. Such groups are deleted from v
 - ▶ Cannot simply test for sum = 0 (why?)



Aggregate Operations (Cont.)

- **Avg**: How to handle average?
 - Maintain **sum** and **count** separately, and divide at the end
- **min, max**: $V = \gamma_{\min(B)}(r)$.
 - Handling insertions on r is straightforward.
 - Maintaining the aggregate values **min** and **max** on deletions may be more expensive. We have to look at the other tuples of r that are in the same group to find the new minimum



Other Operations

- Set intersection: $v = r \cap s$
 - When a tuple is inserted in r , we check if it is present in s , and if so we add it to v . If it already exists, increment count by 1.
 - If the tuple is deleted from r , decrement the count by 1. We delete it from the intersection if it is present there and count=0.
 - Updates to s are symmetric
 - The other set operations, *union* and *set difference* are handled in a similar fashion.
- Outer joins are handled in much the same way as joins but with some extra work



Handling Expressions

- To handle an entire expression, we derive expressions for computing the incremental change to the result of each sub-expressions, starting from the smallest sub-expressions.
- E.g., consider $E_1 \bowtie E_2$ where each of E_1 and E_2 may be a complex expression
 - Suppose the set of tuples to be inserted into E_1 is given by D_1
 - ▶ Computed earlier, since smaller sub-expressions are handled first
 - Then the set of tuples to be inserted into $E_1 \bowtie E_2$ is given by $D_1 \bowtie E_2$
 - ▶ This is just the usual way of maintaining joins



Query Optimization and Materialized Views

- Rewriting queries to use materialized views:
 - A materialized view $v = r \bowtie s$ is available
 - A user submits a query $r \bowtie s \bowtie t$
 - We can rewrite the query as $v \bowtie t$
 - ▶ Whether to do so depends on cost estimates for the two alternative
- Replacing a use of a materialized view by the view definition:
 - A materialized view $v = r \bowtie s$ is available, but without any index on it
 - User submits a query $\sigma_{A=10}(v)$.
 - Suppose also that s has an index on the common attribute B , and r has an index on attribute A .
 - The best plan for this query may be to replace v by $r \bowtie s$, which can lead to the query plan $\sigma_{A=10}(r) \bowtie s$
- Query optimizer should be extended to consider all above alternatives and choose the best overall plan



Materialized View Selection

- **Materialized view selection:** “What is the best set of views to materialize?”
- **Index selection:** “what is the best set of indices to create”
 - closely related to materialized view selection
 - ▶ but simpler
- Materialized view selection and index selection based on typical system **workload** (queries and updates)
 - Typical goal: minimize time to execute workload, subject to constraints on space and time taken for some critical queries/updates
 - One of the steps in database tuning
- Commercial database systems provide tools (called “tuning assistants” or “wizards”) to help the database administrator choose what indices and materialized views to create



Top-K Queries

□ Top-K queries

```
select *  
from r, s  
where r.B = s.B  
order by r.A ascending  
limit 10
```

- Alternative 1: Indexed nested loops join with r as outer
- Alternative 2: estimate highest r.A value in result and add selection (**and** r.A $\leq$ H) to where clause
 - ▶ If < 10 results, retry with larger H



Join Minimization

□ Join minimization

```
select r.A, r.B  
from r, s  
where r.B = s.B
```

□ Check if join with s is redundant, drop it

- E.g., join condition is on foreign key from r to s, r.B is declared as not null, and no selection on s

□ Other sufficient conditions possible

```
select r.A, s2.B  
from r, s as s1, s as s2  
where r.B=s1.B and r.B = s2.B and s1.A < 20 and s2.A < 10
```

- ▶ join with s1 is redundant and can be dropped (along with selection on s1)

□ Lots of research in this area since early times



Multiquery Optimization

□ Example

Q1: **select * from (r natural join t) natural join s**

Q2: **select * from (r natural join u) natural join s**

- Both queries share common subexpression (r natural join s)
- May be useful to compute (r natural join s) once and use it in both queries
 - ▶ But this may be more expensive in some situations
 - e.g. (r natural join s) may be expensive, plans as shown in queries may be cheaper
- **Multiquery optimization**: find best overall plan for a set of queries, exploiting sharing of common subexpressions between queries where it is useful



Multiquery Optimization (Cont.)

- Simple heuristic used in some database systems:
 - optimize each query separately
 - detect and exploit common subexpressions in the individual optimal query plans
 - ▶ May not always give best plan, but is cheap to implement
 - **Shared scans**: widely used special case of multiquery optimization
- Set of materialized views may share common subexpressions
 - As a result, view maintenance plans may share subexpressions
 - Multiquery optimization can be useful in such situations



Optimization of Updates

□ Halloween problem

update R set A = 5 * A
where A > 10

- If index on A is used to find tuples satisfying A > 10, and tuples updated immediately, same tuple may be found (and updated) multiple times
- Solution 1: *Always defer updates*
 - ▶ collect the updates (old and new values of tuples) and update relation and indices in second pass
 - ▶ Drawback: extra overhead even if e.g. update is only on R.B, not on attributes in selection condition
- Solution 2: *Defer only if required*
 - ▶ Perform immediate update if update does not affect attributes in where clause, and deferred updates otherwise.



Parametric Query Optimization

- Example

```
select *  
from r natural join s  
where r.a < $1
```

 - value of parameter \$1 not known at compile time
 - ▶ known only at run time
 - different plans may be optimal for different values of \$1
- Solution 1: optimize at run time, each time query is submitted
 - ▶ can be expensive
- Solution 2: **Parametric Query Optimization:**
 - optimizer generates a set of plans, optimal for different values of \$1
 - ▶ Set of optimal plans usually small for 1 to 3 parameters
 - ▶ Key issue: how to do find set of optimal plans efficiently
 - best one from this set is chosen at run time when \$1 is known
- Solution 3: **Query Plan Caching**
 - If optimizer decides that same plan is likely to be optimal for all parameter values, it caches plan and reuses it, else reoptimize each time
 - Implemented in many database systems



Plan Stability Across Optimizer Changes

- What if 95% of plans are faster on database/optimizer version N+1 than on N, but 5% are slower?
 - Why should plans be slower on new improved optimizer?
 - ▶ Answer: Two wrongs can make a right, fixing one wrong can make things worse!
- Approaches:
 - Allow hints for tuning queries
 - ▶ Not practical for migrating large systems with no access to source code
 - Set optimization level, default to version N (Oracle)
 - ▶ And migrate one query at a time after testing both plans on new optimizer
 - Save plan from version N, and give it to optimizer version N+1
 - ▶ Sybase, XML representation of plans (SQL Server)



Adaptive Query Processing

- Some systems support adaptive operators that change execution algorithm on the fly
 - E.g., (indexed) nested loops join or hash join chosen at run time, depending on size of outer input
- Other systems allow monitoring the behavior of plan at run time and adapt plan
 - E.g., if statistics such as number of rows is found to be very different in reality from what optimizer estimated
 - Can stop execution, compute fresh plan, and restart
 - ▶ But must avoid too many such restarts



Learning to Optimize Query Processing

Learning to optimize query processing

01 Is database a solved problem?

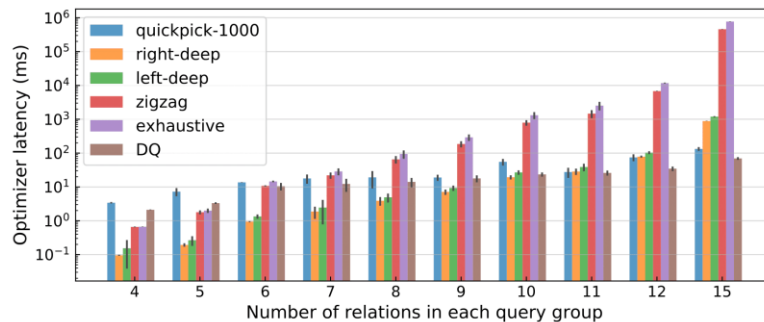
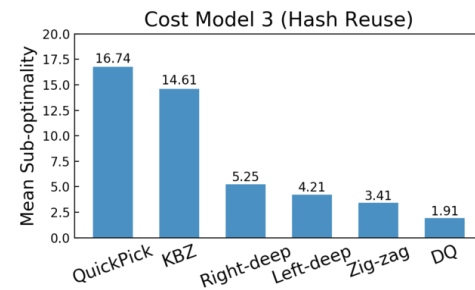
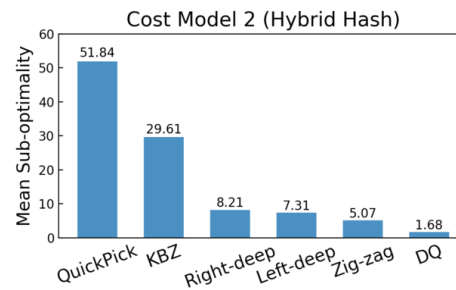
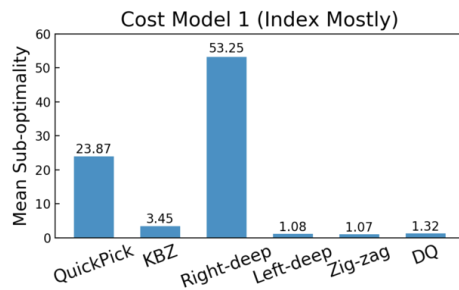
1. Guy Lohman. [Is query optimization a “solved” problem](#). ACM Blog, 2014.
2. Jens Dittrich. [Deep Learning \(m\)eats Databases](#). VLDB 2017 Keynote.

DBMS is a solved problem

- > 40 years of DB research
- We developed
 - fantastic algorithms
 - great systems
 - clever query processing and storage strategies
 - a lot of brilliant stuff
- DBMSs have become fast
 - when you look at TPC-C: we are currently able to execute ~half a million transactions per second
 - to simple index-lookups, e.g., in a hash table, we are currently at 20 million operations per second

DBMS is not solved yet

Take query optimization as an example, DQ outperform **1.32X** of traditional methods



mean sub-optimality of the queries, i.e., “cost(plan from each algorithm) / cost(plan from optimal plan)”, so lower is better

- Sanjay Krishnan et al. [Learning to Optimize Join Queries With Deep Reinforcement Learning](#). arXiv preprint, 2019.

02 Query optimization with RL

1. **DQ:** S. Krishnan, Z. Yang, K. Goldberg, J. Hellerstein, and I. Stoica. [Learning to Optimize Join Queries With Deep Reinforcement Learning](#). arXiv:1808.03196v2, Jan 2019.
2. **Neo:** Ryan Marcus, Parimarjan Negi, Hongzi Mao, Chi Zhang, Mohammad Alizadeh, Tim Kraska, Olga Papaemmanouil, Nesime Tatbul. Neo: A Learned Query Optimizer. PVLDB, 12(11): 1705-1718, 2019.
3. **SkinnerDB:** I. Trummer, S. Moseley, D. Maram, S. Jo, and J. Antonakakis. SkinnerDB: Regret-bounded Query Evaluation via Reinforcement Learning. PVLDB, 11(12):2074–2077, 2018.
4. Xiang Yu, Guoliang Li, Chengliang Chai, Nan Tang. Reinforcement Learning with Tree-LSTM for Join Order Selection. ICDE 2020.

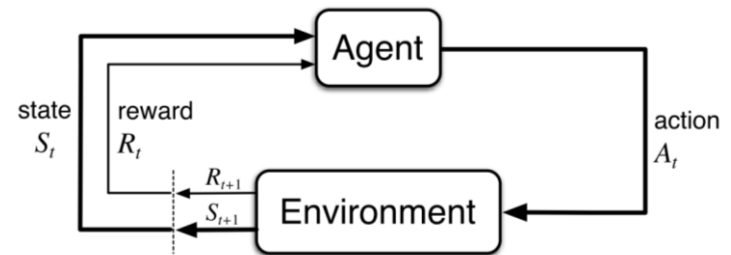
Automatic query optimization

- Traditional query optimization
 - Relational algebraic equivalence
 - Join order enumeration
- Adaptive query processing
 - Traditional optimize-then-execute paradigm
 - Data is changing over time
- Learning to optimize with deep reinforcement learning
 - DQ (Deep Q-learning)
 - Neo (Neural Optimizer)
 - SkinnerDB

Reinforcement learning

- RL Set-Up

- Agents interacts with the environment by *taking actions* and *receiving feedbacks*
- Feedback is in the form of *rewards*
- Agent's utility is defined by the *reward function*
- Must (learn to) act to *maximize expected rewards*



Reinforcement learning

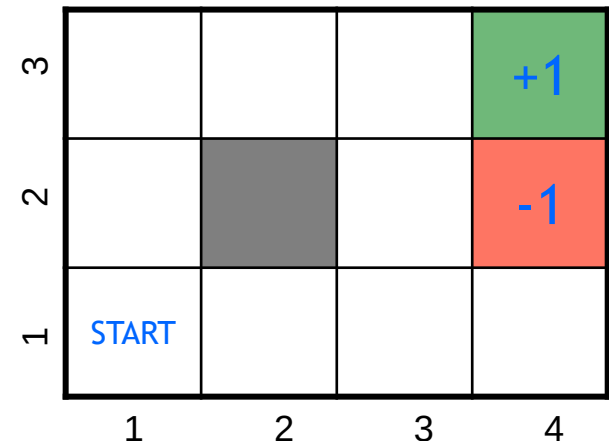
- transitions and rewards usually not available
- how to change the policy based on experience
- how to explore the environment

Markov Decision Process (MDP)

- Set of **states** S , set of **actions** A , initial **state** S_0
- Transition model $P(s,a,s')$
 - $P([1,1], \text{up}, [1,2]) = 0.8$
- Reward function $r(s)$
 - $r([4,3]) = +1$
- Goal: maximize cumulative reward in the long run
- Policy: mapping from S to A
 - $\pi(s)$ or $\pi(s,a)$ (deterministic vs. stochastic)

MDP solvers

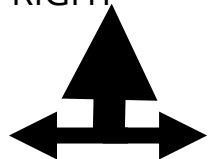
- Dynamic programming
- Monte Carlo methods
- Deep Q-learning



- reward +1 at $[4,3]$, -1 at $[4,2]$
- reward -0.04 for each step
- what's the strategy to achieve max reward?

actions: UP, DOWN, LEFT, RIGHT

80% move UP
10% move LEFT
10% move RIGHT



DQ: optimize join with deep Q-learning

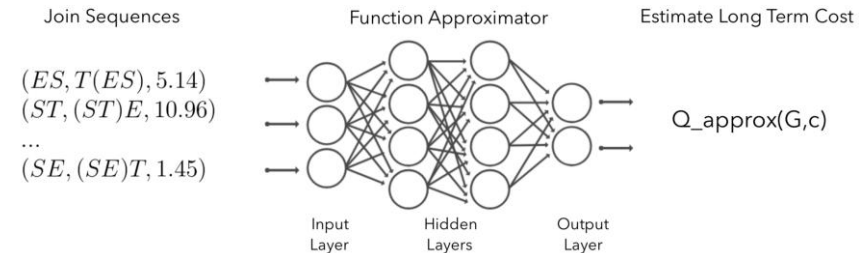
RL can be modeled as a Markov Decision Process (MDP)

Formulate join ordering as an MDP (G, c, G', J)

- **States G** : the remaining relations to be joined.
- **Actions c** : a valid join out of the remaining relations.
- **Next states G'** : naturally, this is the old “remaining relations” set with two relations removed and their resultant join added.
- **Reward J** : estimated cost of the new join.

Apply Q-learning to solve the join-ordering MDP

- Q-function: $Q(G, c) = J(c) + \min_{c'} Q(G', c')$
- describes the long-term cost of each join: the **cumulative cost** if we act optimally for all subsequent joins after the current join decision.



Learning algorithm:

- (1) Start with the initial query graph,
- (2) Find the join with the lowest $Q(G, c)$,
- (3) Update the query graph and repeat.

Training the optimizer in DQ

To learn the Q-function (Training the model)

1. **Training data**: To learn the Q-function we first need to observe past execution data. DQ can accept a list of (G, c, G', J) from any underlying optimizer (**run a classical left-deep dynamic program**).

For example: $(G, c, G', J) = (\{E, S, T\}, \text{join}(S, T), \{E, ST\}, 110)$ means that we start with the initial query graph (**state**) and joining S and T together (**action**).

2. Featurization of **states** and **actions**

Feed states G and actions c into the network as **fixed-length feature vectors**

1-hot vectors to encode: (1) the set of all attributes A_G present in the query graph, out of all attributes in the schema, (2) the participating attributes A_L from left side of the join, and (3) those A_R from right side of the join

```
SELECT *
FROM Emp, Pos, Sal
WHERE Emp.rank =
      Pos.rank
AND Pos.code = Sal.code
```

(a) Example query

$$A_G = [E.id, E.name, E.rank, \\ P.rank, P.title, P.code, \\ S.code, S.amount] \\ = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1]$$

(b) Query graph featurization

$$A_L = [E.id, E.name, E.rank] \\ = [1 \ 1 \ 1 \ 0 \ 0 \ 0 \ 0 \ 0] \\ A_R = [P.rank, P.title, P.code] \\ = [0 \ 0 \ 0 \ 1 \ 1 \ 1 \ 0 \ 0]$$

(c) Features of $E \bowtie P$

$$A_L = [E.id, E.name, E.rank, \\ P.rank, P.title, P.code] \\ = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 0 \ 0] \\ A_R = [S.code, S.amount] \\ = [0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1]$$

(d) Features of $(E \bowtie P) \bowtie S$

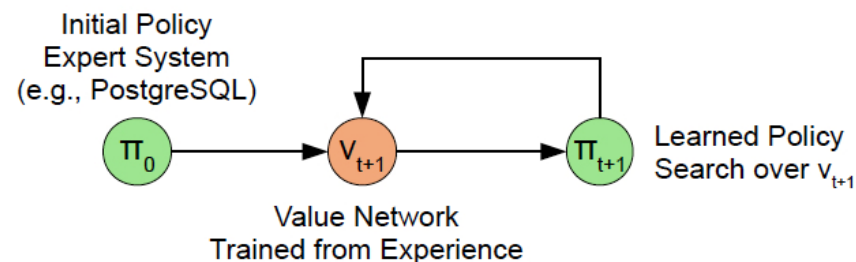
Neo: a learned query optimizer

DQ

- Only optimize the join ordering
- DQ uses a fully connected NN to approximate the Q-function
- The model is trained with a standard stochastic gradient descent (SGD) algorithm
- DQ train the optimizer from scratch

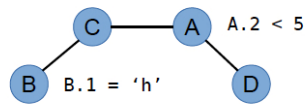
Neo

- Neo optimize join ordering and execution plan
- Neo use a tree CNN instead of fully connected NN used by DQ
- The model was trained via value iteration
- Neo bootstraps its query optimization model from existing optimizers (PostgreSQL's)



Neo (cont.)

SELECT * FROM A, B, C, D WHERE
A.3=C.3 AND A.4=D.4 AND C.5=B.5
AND A.2<5 AND B.1='h';



	A	B	C	D	E
A	0	0	1	1	0
B	0	0	1	0	0
C	1	1	0	0	0
D	1	0	0	0	0
E	0	0	0	0	0

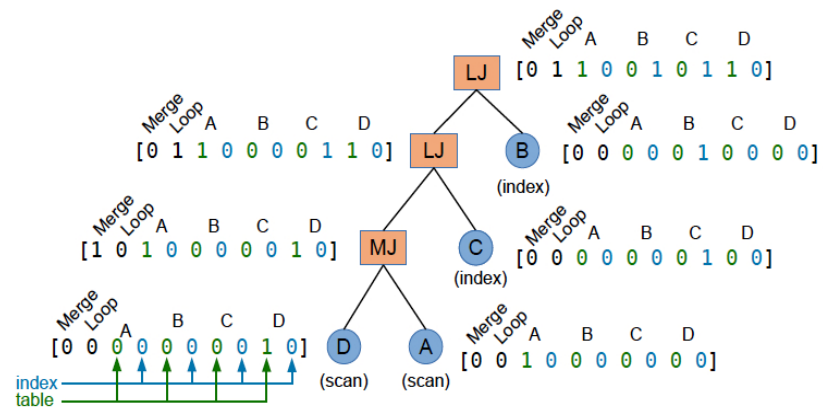
Join Graph

A.1	A.2	...	B.1	B.2	...	E.1	E.2
0	1	...	1	0	...	0	0

Column Predicates

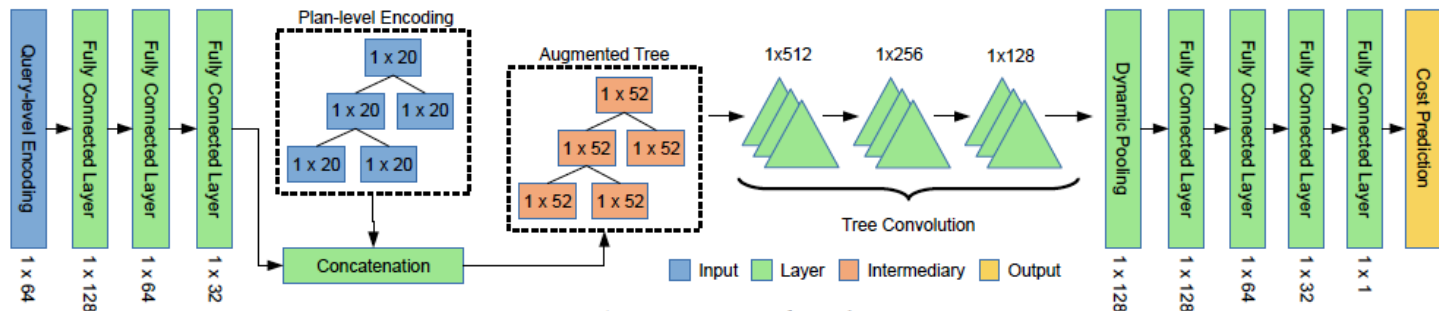
[0 1 1 0 1 0 0 0 0 0 0 1 ... 1 0 ... 0 0]

Query-level encoding



Plan-level encoding

Value network (tree CNN)



SkinnerDB: learning by doing

ML for query optimization:

Inter-query learning: DQ, Neo

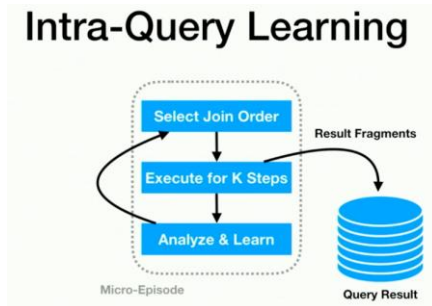
- 🔗 Train the query optimizer with past queries

Intra-query learning: SkinnerDB

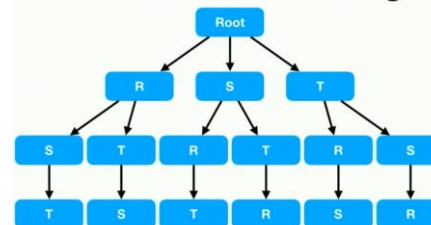
- 🔗 Learn optimal join orders on the fly, during the execution of the current query
- 🔗 Divide the execution of a query into many small time slices
- 🔗 Different join orders are tried in different time slices (UCT search tree)
- 🔗 Merge result tuples generated according to different join orders until a complete result is obtained
- 🔗 SkinnerDB can converge to optimal join orders with regret bounds

	<i>Inter-query</i> learning	<i>Intra-query</i> learning
Training	Past queries	Current query
Application	Current query	Current query

Intra-Query Learning



UCT for Join Ordering



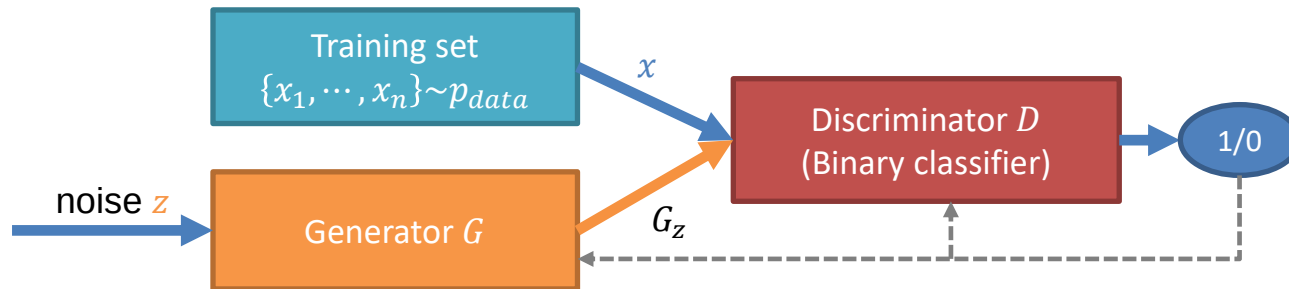
04 Adversarial training for query optimizer

1. Generative adversarial networks
2. Monte Carlo tree search.
3. AlphaGo : D. Silver, et al. Mastering the game of go with deep neural networks and tree search. In Nature. Nature Research, 2016.

Shortcomings of the existing methods

- **Shortcoming 1:** lack of training data
 - DQ learns from scratch and takes very long time for training
 - Neo relies on the existing knowledge but with limited improvement
 - SkinnerDB tries to learn an adaptive execution by dividing the data into many small slots and each slot acts a distinct execution plan
- **Shortcoming 2:** Lack of capability to tradeoff exploitation and exploration
 - DQ relies on deep Q-learning tries to exhaust all enumerations which is impossible for large queries
 - Neo relies on value iteration which takes a long time to converge
- **Our solution**
 - Generative adversarial neural network (GAN) for the shortcoming 1
 - Monte Carlo search tree (MCTS) to address the shortcoming 2

Generative adversarial network (GAN)



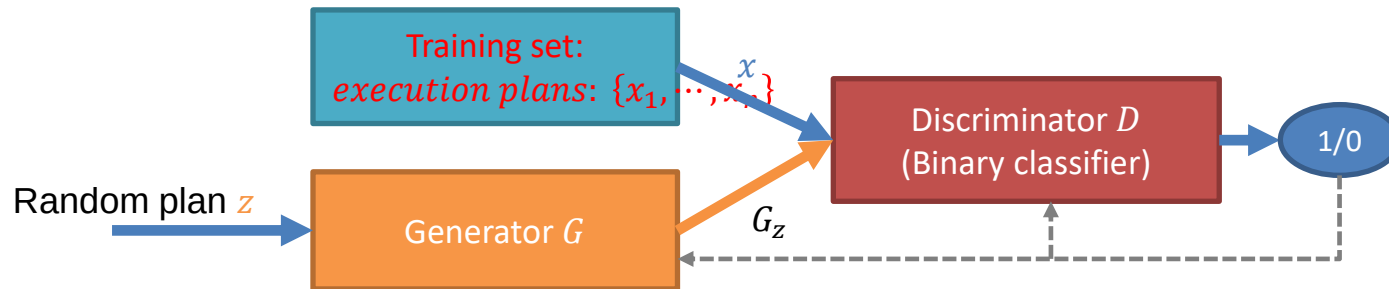
MinMax Game (Zero-sum gaming)

- Generator tries to fool discriminator (i.e. generate realistic samples)
- Discriminator tries to distinguish fake from real samples
- Each tries to minimize the objective function maximized by the other

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

[Goodfellow et al., 2014]

Query optimization via adversarial training



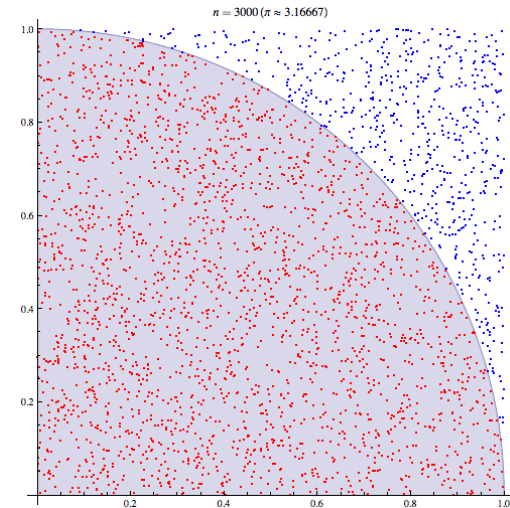
MinMax Game

- The discriminator D is a cascaded joins of value network connecting all tables together
- The generator G is a cascaded series of the MCTS improved policy (i.e. generate realistic samples)
- Each tries to minimize the objective function maximized by the other

$$\min_G \max_D (D, G) = \mathbb{E}_x [\log D(x)] + \mathbb{E}_z [\log(1 - D(G(z)))]$$

Monte Carlo Tree Search (MCTS)

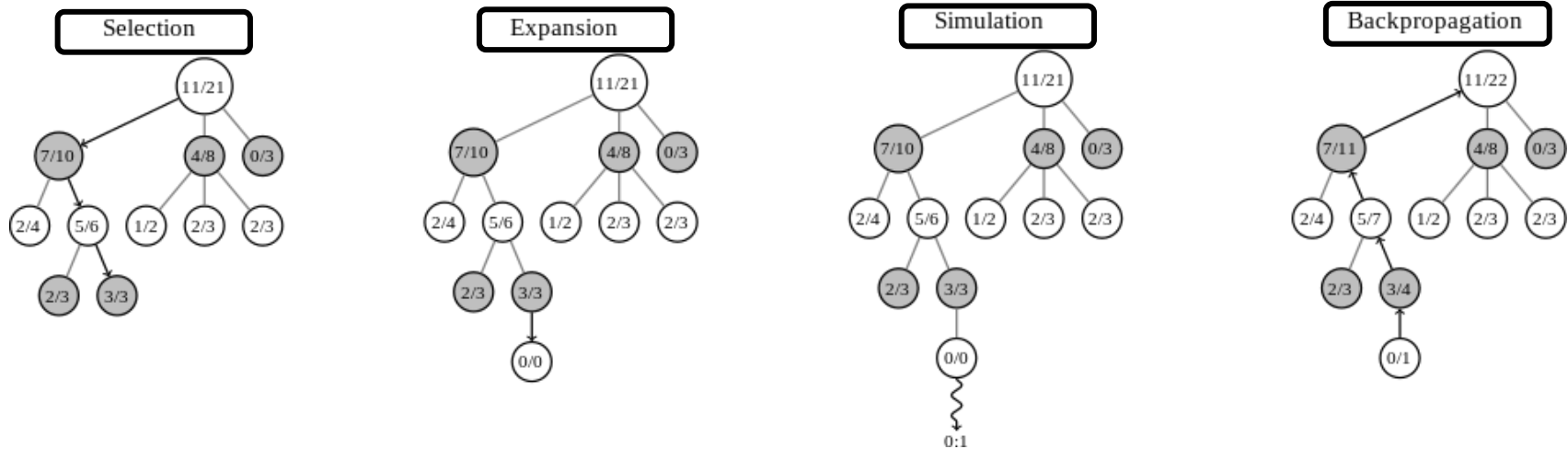
- Monte Carlo Experiments : repeated random sampling to obtain numerical results
- Search method
- Method for making optimal decisions in artificial intelligence (AI) problems
- The strongest Go AIs (Fuego, Pachi, Zen, and Crazy Stone) all rely on MCTS



Monte Carlo Tree Search

Each round of MCTS consists of four steps

1. **Selection**: start from root R and select successive child nodes until a leaf node L is reached
2. **Expansion**: create one (or more) child nodes and choose node C from one of them
3. **Simulation**: play a random rollout (choosing a uniform random move) from node C
4. **Backpropagation**: update information in the nodes on the path from C to R



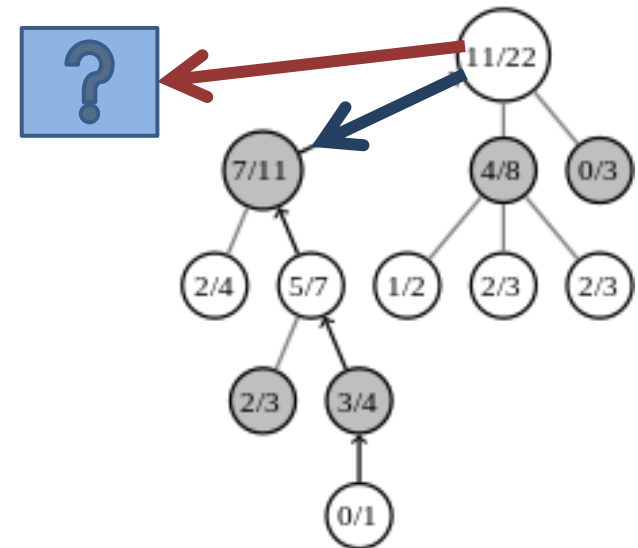
MCTS – Upper Confidence Bounds for Trees

- For every child we calculate the following function to tradeoff exploration and exploitation
- Convergence to the optimal solution
 - Kocsis, L. & Szepesvári, C. Bandit based Monte-Carlo planning (2006)

Exploitation **Exploration**

$$\frac{\omega_i}{n_i} + c \cdot \sqrt{\frac{\ln(t)}{n_i}}$$

W_i #wins after visiting the node i
 n_i #times node i has been visited
 C exploration parameter
 t #times node i parent has been visited



Experimental studies

x	o	x
o	x	o
x	o	x

A tik-tac-toe game with 10 columns and 10 rows

- The training stops when agent win over 80%

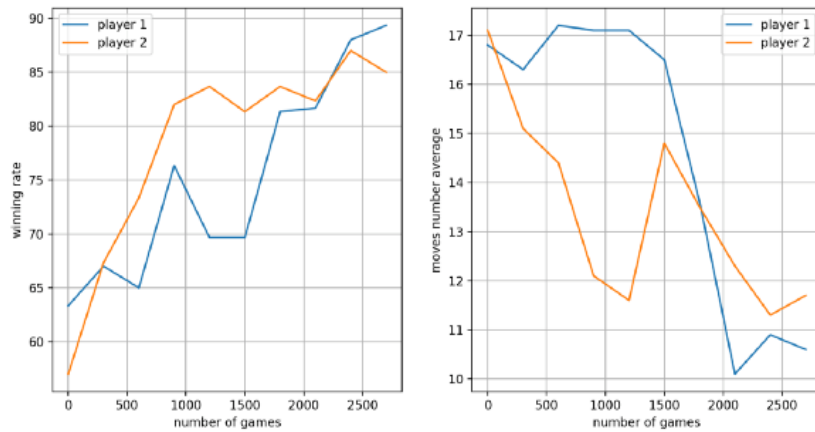
Learning process of Deep Q-Learning:

1. Offline learning with planning, where batches of state-action pairs were stored and the network was trained on these batches
2. The playing policy was an ϵ -greedy policy which means that an exploration factor ϵ was chosen
3. Take ReLu function as an activation function
4. The agent played 300 episodes against random player and measured the winning rate

Learning process of MCTS:

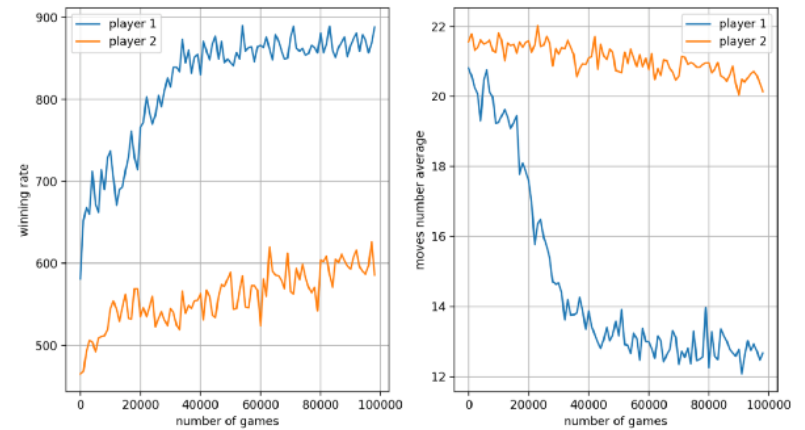
1. The agent takes 1M episodes, but the learning is a lot faster (a few hours)
2. To store a tree of 1M episodes it takes around 800MB and it is growing when the agent continues learning. So it is a lot more than the CNN which stayed constant through all the learning process.
3. The graph below shows the winning rate of the MCTS agent against random player as the learning progresses.

Experimental studies



Deep Q-learning

Player 1: DQN agent
Player 2: random agent



MCTS

Player 1: MCTS agent
Player 2: random agent